Laboratorium Task 3

by Paweł Paduch (https://achilles.tu.kielce.pl/portal/author/332bd2d158a24964b2b98a7fc2879842) published 2 years ago, last modified 2 years ago

1. **Zaczynamy [0,5 pkt]**
   - **Utworzyć nową solucję a w niej projekt typu konsolowego .net core**
   - **Zademonstrować działanie poniższej funkcji:**

```
private static Task<int[]> calculateSomethingAsync(int size)
{
    return Task.Factory.StartNew<int[]>((s) => {
        if (s==null) { throw new ArgumentNullException(nameof(s)); }
        int[] result = new int[(int)s];
        for (int i = 0; i < (int)s; i++)
        {
            Thread.SpinWait(30000000);
            result[i] = Random.Shared.Next(10);
            Console.WriteLine($"I calculated element {i} as [{result[i]}]");
        }
        return result;
    },size);
}
```

2. **Raportowanie postępu [0,5 pkt.]**
   - **Przeciążyć powyższą funkcję tak by zamiast napisu "I calculated element..." raportowała za pomocą interfejsu** `IProgress<(int,int)>` **.**
   - **Uwzględnić możliwość podania null zamiast Progress.**
   - **Zademonstrować działanie dodając obsługę eventu** `ProgressChanged` **wypisując numer iteracji i wylosowaną wartość .**

3. **Anulowanie [0,5 pkt.]**
   - **Do powyższej metody dodać możliwość anulowania za pomocą** `CancellationToken`
   - **Metoda powinna sprawdzać przy każdej iteracji czy nastąpiło żądanie anulowania i rzucić wyjątkiem wraz z odpowiednim komunikatem, np. "Nastąpiło żądanie anulowania".**
   - **Uruchomić aplikację i przetestować anulowanie używając** `CancelAfter(5000)` **(z obiekt klasy** `CancellationTokenSource` **). Dzięki temu zażądamy anulowania po upływie 5 sekund.**
   - **W Main przechwycić wygenerowany wyjątek. Można użyć AggregateException.Flatten().**

4. **Użycie *continueWith* opcjonalne [0,5 pkt.]**
   - **na utworzonym tasku z metody** `calculateSomethingAsync` **wykonać** `ContinueWith` **w przypadku błędu (** `TaskContinuationOptions.NotOnRanToCompletion` **). Utworzy nam się alternatywny task, nazwijmy go onFaultTask.**
   - **Wypisać błąd przekazany od poprzednika**
   - **podobnie przy pomocy** `ContinueWith` **stworzyć task wyświetlający zawartość zwróconej tablicy ale tylko w przypadku poprawnego zakończenia (** `TaskContinuationOptions.OnlyOnRanToCompletion` **)**
   - **Poczekać na oba taski za pomocą** `Task.WaitAll` **. Tu wymagane utworzenie tablicy obu tasków.**
   - **Przechwycić wyjątek rzucony przez task alternatywny.**
   - **Przetestować przez zakomentowanie/odkomentowanie** `CancelAfter(5000)`

5. **Wykorzystanie *ParallelFor [0,5 pkt]***
   - **Założyć nowy projekt**

- Utworzyć prosty jednowątkowy program w którym generujemy tablicę z 10 mln losowych elementów
- Można posłużyć się metodą

```
static int[] GenerateRandomArray(int size)
{
    Random random = new Random();
    int[] array = new int[size];
    for (int i = 0; i < size; i++)
    {
        array[i] = random.Next(10);
    }
    return array;
}
```

- Zsumuj wszystkie elementy za pomocą jednowątkowego wyrażenia *Linq*
  `foreach (int number in numbers) sum += number;`
- Zmierz czas wykonania

6. **Na podstawie powyższego przeprowadź eksperymenty z sumowaniem równoległym za pomocą** [1 pkt.]
   - `Parallel.ForEach(numbers, number => sum += number);` Zmierz czas i sprawdź czy suma jest identyczna
   - Zabezpiecz sumowanie za pomocą lock. Zmierz czas, sprawdź poprawność wyników.
   - Wykonaj sumowanie za pomocą jednowątkowej metody Linq Sum() `numbers.Sum(number => number);` Zmierz czas, sprawdź poprawność wyników.
   - Wykonaj to samo dla zapytania *PLINQ* `numbers.AsParallel().ForAll(number => sum += number);` Zmierz czas, sprawdź poprawność wyników.
   - Zabezpiecz powyższe wyliczenia za pomocą lock. Zmierz czas, sprawdź poprawność wyników.
   - Wykonaj sumowanie za pomocą sumy z *PLINQ* na dwa sposoby:
     - `numbers.AsParallel().Sum(number => number);`
     - `numbers.AsParallel().Sum();`
   - Zmierz czas i sprawdź poprawność wyników. Jednak zrób to uruchamiając program najpierw z pierwszym przypadkiem potem z drugim a następnie z obydwoma. Porównaj wyniki.

7. **Wykonaj sumowanie na zwykłych taskach** [0,5 pkt.]
   - Do odczytania liczby procesorów użyj `Environment.ProcessorCount`
   - Poczekaj na zakończenie tasków i zsumuj od nich wyniki.
     `double[] partialResults = Task.WhenAll(tasks).Result;`
     `sum = partialResults.Sum();`
   - Oczywiście wykonaj pomiary czasu i zweryfikuj poprawność danych.

8. **Sumowanie z obliczaniem** [1 pkt.]
   - Wykonaj wszystkie testy dla przykładu z punktu 6 oraz 7, ale zsumuj wyniki funkcji

```
static double calculatedValue(double value)
{
    return (Math.Sin(Math.Sin(value * value) * Math.Cos(value * value) / Math.PI)
}
```

   - Np. Dla operacji jednowątkowej:

```
foreach (int number in numbers)
{
    sum += calculatedValue(number);
}
```